

Chương 19 – Huấn luyện và Triển khai các Mô hình TensorFlow ở Quy mô lớn

Notebook này chứa toàn bộ mã nguồn mẫu và lời giải cho các bài tập trong chương 19.

 [Open in Colab](#)  [Open in Kaggle](#)

Thiết lập

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Cảnh báo: Các phiên bản TensorFlow mới nhất dựa trên Keras 3. Đối với các chương 10-15, việc cập nhật mã nguồn để hỗ trợ Keras 3 không quá khó, nhưng đối với chương này thì khó hơn nhiều, vì vậy tôi đã phải quay lại sử dụng Keras 2. Để làm điều đó, tôi đặt biến môi trường `TF_USE_LEGACY_KERAS` thành `"1"` và nhập gói `tf.keras`. Điều này đảm bảo rằng `tf.keras` trỏ đến `tf.keras`, chính là Keras 2.*.

```
IS_COLAB = "google.colab" in sys.modules
if IS_COLAB:
    import os
    os.environ["TF_USE_LEGACY_KERAS"] = "1"
    import tf.keras
```

Và TensorFlow ≥ 2.8 :

```
from packaging import version
import tensorflow as tf

assert version.parse(tf.__version__) >= version.parse("2.8.0")
```

Nếu chạy trên Colab hoặc Kaggle, bạn cần cài đặt thư viện client Google AI Platform, thư viện này sẽ được sử dụng sau này trong notebook. Bạn có thể bỏ qua các cảnh báo về không tương thích phiên bản.

- **Cảnh báo:** Trên Colab, bạn phải khởi động lại Runtime sau khi cài đặt, và tiếp tục với các ô tiếp theo.

```
import sys
if "google.colab" in sys.modules or "kaggle_secrets" in sys.modules:
    %pip install -q -U google-cloud-aiplatform
```

Chương này thảo luận về cách chạy hoặc huấn luyện một mô hình trên một hoặc nhiều GPU, vì vậy hãy đảm bảo có ít nhất một GPU, nếu không sẽ đưa ra cảnh báo:

```
if not tf.config.list_physical_devices('GPU'):
    print("No GPU was detected. Neural nets can be very slow without a GPU.")
    if "google.colab" in sys.modules:
        print("Go to Runtime > Change runtime and select a GPU hardware "
              "accelerator.")
    if "kaggle_secrets" in sys.modules:
        print("Go to Settings > Accelerator and select GPU.")
```

Phục vụ Mô hình TensorFlow (Serving)

Hãy bắt đầu bằng cách triển khai một mô hình sử dụng TF Serving, sau đó chúng ta sẽ triển khai lên Google Vertex AI.

Sử dụng TensorFlow Serving

Điều đầu tiên chúng ta cần làm là xây dựng, huấn luyện một mô hình và xuất nó sang định dạng SavedModel.

✧ Xuất các SavedModel

Hãy tải tập dữ liệu MNIST, chuẩn hóa và chia nhỏ nó.

```
from pathlib import Path
import tensorflow as tf

# extra code - load and split the MNIST dataset
mnist = tf.keras.datasets.mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = mnist
X_valid, X_train = X_train_full[:5000], X_train_full[5000:]
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]

# extra code - build & train an MNIST model (also handles image preprocessing)
tf.random.set_seed(42)
tf.keras.backend.clear_session()
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28], dtype=tf.uint8),
    tf.keras.layers.Rescaling(scale=1 / 255),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
              metrics=["accuracy"])
model.fit(X_train, y_train, epochs=10, validation_data=(X_valid, y_valid))

model_name = "my_mnist_model"
model_version = "0001"
model_path = Path(model_name) / model_version
model.save(model_path, save_format="tf")
```

```
Epoch 1/10
1719/1719 [=====] - 2s 1ms/step - loss: 0.7012 - accuracy: 0.8241 - val_loss: 0.3715 - val_accuracy: 0.
Epoch 2/10
1719/1719 [=====] - 2s 943us/step - loss: 0.3536 - accuracy: 0.9020 - val_loss: 0.2990 - val_accuracy:
Epoch 3/10
1719/1719 [=====] - 2s 933us/step - loss: 0.3036 - accuracy: 0.9145 - val_loss: 0.2651 - val_accuracy:
Epoch 4/10
1719/1719 [=====] - 2s 965us/step - loss: 0.2736 - accuracy: 0.9231 - val_loss: 0.2436 - val_accuracy:
Epoch 5/10
1719/1719 [=====] - 2s 946us/step - loss: 0.2509 - accuracy: 0.9296 - val_loss: 0.2257 - val_accuracy:
Epoch 6/10
1719/1719 [=====] - 2s 974us/step - loss: 0.2322 - accuracy: 0.9350 - val_loss: 0.2121 - val_accuracy:
Epoch 7/10
1719/1719 [=====] - 2s 959us/step - loss: 0.2161 - accuracy: 0.9400 - val_loss: 0.1970 - val_accuracy:
Epoch 8/10
1719/1719 [=====] - 2s 944us/step - loss: 0.2021 - accuracy: 0.9432 - val_loss: 0.1880 - val_accuracy:
Epoch 9/10
1719/1719 [=====] - 2s 945us/step - loss: 0.1898 - accuracy: 0.9470 - val_loss: 0.1778 - val_accuracy:
Epoch 10/10
1719/1719 [=====] - 2s 940us/step - loss: 0.1793 - accuracy: 0.9494 - val_loss: 0.1685 - val_accuracy:
INFO:tensorflow:Assets written to: my_mnist_model/0001/assets
```

Hãy xem cấu trúc thư mục tệp (chúng ta đã thảo luận về công dụng của từng tệp này trong chương 10):

```
sorted([str(path) for path in model_path.parent.glob("**/*")]) # extra code
```

```
['my_mnist_model/0001',
 'my_mnist_model/0001/assets',
 'my_mnist_model/0001/keras_metadata.pb',
 'my_mnist_model/0001/saved_model.pb',
 'my_mnist_model/0001/variables',
 'my_mnist_model/0001/variables/variables.data-00000-of-00001',
 'my_mnist_model/0001/variables/variables.index']
```

Hãy kiểm tra SavedModel:

```
!saved_model_cli show --dir '{model_path}'
```

```
The given SavedModel contains the following tag-sets:
'serve'
```

```
!saved_model_cli show --dir '{model_path}' --tag_set serve
```

```
The given SavedModel MetaGraphDef contains SignatureDefs with the following keys:
SignatureDef key: "__saved_model_init_op"
SignatureDef key: "serving_default"
```

```
!saved_model_cli show --dir '{model_path}' --tag_set serve \
    --signature_def serving_default
```

```
The given SavedModel SignatureDef contains the following input(s):
inputs['flatten_input'] tensor_info:
  dtype: DT_UINT8
  shape: (-1, 28, 28)
  name: serving_default_flatten_input:0
The given SavedModel SignatureDef contains the following output(s):
outputs['dense_1'] tensor_info:
  dtype: DT_FLOAT
  shape: (-1, 10)
  name: StatefulPartitionedCall:0
Method name is: tensorflow/serving/predict
```

Để biết thêm chi tiết, bạn có thể chạy lệnh sau:

```
!saved_model_cli show --dir '{model_path}' --all
```

▼ Cài đặt và Khởi chạy TensorFlow Serving

Nếu bạn đang chạy notebook này trong Colab hoặc Kaggle, TensorFlow Server cần phải được cài đặt:

```
if "google.colab" in sys.modules or "kaggle_secrets" in sys.modules:
    url = "https://storage.googleapis.com/tensorflow-serving-apt"
    src = "stable tensorflow-model-server tensorflow-model-server-universal"
    !echo 'deb {url} {src}' > /etc/apt/sources.list.d/tensorflow-serving.list
    !curl '{url}/tensorflow-serving.release.pub.gpg' | apt-key add -
    !apt update -q && apt-get install -y tensorflow-model-server
    %pip install -q -U tensorflow-serving-api
```

Nếu `tensorflow_model_server` đã được cài đặt (ví dụ: nếu bạn đang chạy notebook này trong Colab), thì 2 ô tiếp theo sẽ khởi động server. Nếu hệ điều hành của bạn là Windows, bạn có thể cần chạy lệnh `tensorflow_model_server` trong terminal và thay thế `${MODEL_DIR}` bằng đường dẫn đầy đủ đến thư mục `my_mnist_model`.

```
import os

os.environ["MODEL_DIR"] = str(model_path.parent.absolute())
```

```
%%bash --bg
tensorflow_model_server \
  --port=8500 \
  --rest_api_port=8501 \
  --model_name=my_mnist_model \
  --model_base_path="${MODEL_DIR}" >my_server.log 2>&1
```

```
import time

time.sleep(2) # let's wait a couple seconds for the server to start
```

Nếu bạn đang chạy notebook này trên máy cá nhân và ưu tiên cài đặt TF Serving bằng Docker, trước tiên hãy đảm bảo [Docker](#) đã được cài đặt, sau đó chạy các lệnh sau trong terminal. Bạn phải thay thế `/path/to/my_mnist_model` bằng đường dẫn tuyệt đối phù hợp đến thư mục `my_mnist_model`, nhưng không sửa đổi đường dẫn container `/models/my_mnist_model`.

```
docker pull tensorflow/serving # tải ảnh TF Serving mới nhất
```

```
docker run -it --rm -v "/path/to/my_mnist_model:/models/my_mnist_model" \
-p 8500:8500 -p 8501:8501 -e MODEL_NAME=my_mnist_model tensorflow/serving
```

✓ Truy vấn TF Serving thông qua REST API

Tiếp theo, hãy gửi một truy vấn REST tới TF Serving:

```
import json

X_new = X_test[:3] # pretend we have 3 new digit images to classify
request_json = json.dumps({
    "signature_name": "serving_default",
    "instances": X_new.tolist(),
})
```

```
request_json[:100] + "..." + request_json[-10:]
```

```
'{"signature_name": "serving_default", "instances": [[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0..., 0, 0]]]}'
```

Bây giờ hãy sử dụng REST API của TensorFlow Serving để thực hiện dự đoán:

```
import requests

server_url = "http://localhost:8501/v1/models/my_mnist_model:predict"
response = requests.post(server_url, data=request_json)
response.raise_for_status() # raise an exception in case of error
response = response.json()
```

```
import numpy as np

y_proba = np.array(response["predictions"])
y_proba.round(2)

array([[0. , 0. , 0. , 0. , 0. , 0. , 0. , 1. , 0. , 0. ],
       [0. , 0. , 0.99, 0.01, 0. , 0. , 0. , 0. , 0. , 0. ],
       [0. , 0.97, 0.01, 0. , 0. , 0. , 0. , 0.01, 0. , 0. ]])
```

✓ Truy vấn TF Serving thông qua gRPC API

```
from tensorflow_serving.apis.predict_pb2 import PredictRequest

request = PredictRequest()
request.model_spec.name = model_name
request.model_spec.signature_name = "serving_default"
input_name = model.input_names[0] # == "flatten_input"
request.inputs[input_name].CopyFrom(tf.make_tensor_proto(X_new))
```

```
import grpc
from tensorflow_serving.apis import prediction_service_pb2_grpc

channel = grpc.insecure_channel('localhost:8500')
predict_service = prediction_service_pb2_grpc.PredictionServiceStub(channel)
response = predict_service.Predict(request, timeout=10.0)
```

Chuyển đổi phản hồi thành một tensor:

```
output_name = model.output_names[0]
outputs_proto = response.outputs[output_name]
y_proba = tf.make_ndarray(outputs_proto)
```

```
y_proba.round(2)

array([[0. , 0. , 0. , 0. , 0. , 0. , 0. , 1. , 0. , 0. ],
       [0. , 0. , 0.99, 0.01, 0. , 0. , 0. , 0. , 0. , 0. ],
       [0. , 0.97, 0.01, 0. , 0. , 0. , 0. , 0.01, 0. , 0. ]],
      dtype=float32)
```

Nếu máy khách của bạn không bao gồm thư viện TensorFlow, bạn có thể chuyển đổi phản hồi thành mảng NumPy như sau:

```
# extra code - shows how to avoid using tf.make_ndarray()
output_name = model.output_names[0]
outputs_proto = response.outputs[output_name]
shape = [dim.size for dim in outputs_proto.tensor_shape.dim]
y_proba = np.array(outputs_proto.float_val).reshape(shape)
y_proba.round(2)

array([[0. , 0. , 0. , 0. , 0. , 0. , 0. , 1. , 0. , 0. ],
       [0. , 0. , 0.99, 0.01, 0. , 0. , 0. , 0. , 0. , 0. ],
       [0. , 0.97, 0.01, 0. , 0. , 0. , 0. , 0.01, 0. , 0. ]])
```

▼ Triển khai một phiên bản mô hình mới

```
# extra code - build and train a new MNIST model version
np.random.seed(42)
tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28], dtype=tf.uint8),
    tf.keras.layers.Rescaling(scale=1 / 255),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
              metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=10,
                   validation_data=(X_valid, y_valid))
```

```
Epoch 1/10
1719/1719 [=====] - 2s 931us/step - loss: 0.7039 - accuracy: 0.8056 - val_loss: 0.3418 - val_accuracy:
Epoch 2/10
1719/1719 [=====] - 1s 855us/step - loss: 0.3204 - accuracy: 0.9082 - val_loss: 0.2674 - val_accuracy:
Epoch 3/10
1719/1719 [=====] - 2s 883us/step - loss: 0.2650 - accuracy: 0.9235 - val_loss: 0.2227 - val_accuracy:
Epoch 4/10
1719/1719 [=====] - 1s 869us/step - loss: 0.2319 - accuracy: 0.9329 - val_loss: 0.2032 - val_accuracy:
Epoch 5/10
1719/1719 [=====] - 1s 870us/step - loss: 0.2089 - accuracy: 0.9399 - val_loss: 0.1833 - val_accuracy:
Epoch 6/10
1719/1719 [=====] - 1s 871us/step - loss: 0.1908 - accuracy: 0.9446 - val_loss: 0.1740 - val_accuracy:
Epoch 7/10
1719/1719 [=====] - 2s 873us/step - loss: 0.1756 - accuracy: 0.9490 - val_loss: 0.1605 - val_accuracy:
Epoch 8/10
1719/1719 [=====] - 2s 877us/step - loss: 0.1631 - accuracy: 0.9524 - val_loss: 0.1543 - val_accuracy:
Epoch 9/10
1719/1719 [=====] - 2s 879us/step - loss: 0.1517 - accuracy: 0.9567 - val_loss: 0.1460 - val_accuracy:
Epoch 10/10
1719/1719 [=====] - 1s 872us/step - loss: 0.1429 - accuracy: 0.9584 - val_loss: 0.1358 - val_accuracy:
```

```
model_version = "0002"
model_path = Path(model_name) / model_version
model.save(model_path, save_format="tf")
```

```
INFO:tensorflow:Assets written to: my_mnist_model/0002/assets
```

Hãy xem lại cấu trúc thư mục tệp một lần nữa:

```
sorted([str(path) for path in model_path.parent.glob("**/*")]) # extra code
```

```
['my_mnist_model/0001',
 'my_mnist_model/0001/assets',
 'my_mnist_model/0001/keras_metadata.pb',
 'my_mnist_model/0001/saved_model.pb',
 'my_mnist_model/0001/variables',
 'my_mnist_model/0001/variables/variables.data-00000-of-00001',
 'my_mnist_model/0001/variables/variables.index',
 'my_mnist_model/0002',
 'my_mnist_model/0002/assets',
 'my_mnist_model/0002/keras_metadata.pb',
 'my_mnist_model/0002/saved_model.pb',
 'my_mnist_model/0002/variables',
```

```
'my_mnist_model/0002/variables/variables.data-00000-of-00001',  
'my_mnist_model/0002/variables/variables.index']
```

Cảnh báo: Bạn có thể cần đợi một phút trước khi mô hình mới được TensorFlow Serving tải lên.

```
import requests  
  
server_url = "http://localhost:8501/v1/models/my_mnist_model:predict"  
  
response = requests.post(server_url, data=request_json)  
response.raise_for_status()  
response = response.json()
```

```
response.keys()
```

```
dict_keys(['predictions'])
```

```
y_proba = np.array(response["predictions"])  
y_proba.round(2)
```

```
array([[0. , 0. , 0. , 0. , 0. , 0. , 0. , 1. , 0. , 0. ],  
       [0. , 0. , 0.99, 0.01, 0. , 0. , 0. , 0. , 0. , 0. ],  
       [0. , 0.99, 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ]])
```

▼ Tạo Dịch vụ Dự đoán trên Vertex AI

Làm theo hướng dẫn trong sách để tạo tài khoản Google Cloud Platform và kích hoạt Vertex AI và Cloud Storage API. Sau đó, nếu bạn đang chạy notebook này trong Colab, bạn có thể chạy ô sau để xác thực bằng cùng một tài khoản Google mà bạn đã sử dụng với Google Cloud Platform và cho phép Colab này truy cập dữ liệu của bạn.

CẢNH BÁO: chỉ làm điều này nếu bạn tin tưởng notebook này!

- Hãy cực kỳ cẩn thận nếu đây không phải là notebook chính thức từ <https://github.com/ageron/handson-ml3>: URL Colab phải bắt đầu bằng <https://colab.research.google.com/github/ageron/handson-ml3>. Nếu không, mã nguồn có thể làm bất cứ điều gì với dữ liệu của bạn.

Nếu bạn không chạy notebook này trong Colab, bạn phải làm theo hướng dẫn trong sách để tạo một service account và tạo khóa cho nó, tải nó về thư mục của notebook này và đặt tên là `my_service_account_key.json` (hoặc đảm bảo biến môi trường

`GOOGLE_APPLICATION_CREDENTIALS` trỏ đến khóa của bạn).

```
project_id = "my_project" ##### CHANGE THIS TO YOUR PROJECT ID #####  
  
if "google.colab" in sys.modules:  
    from google.colab import auth  
    auth.authenticate_user()  
elif "kaggle_secrets" in sys.modules:  
    from kaggle_secrets import UserSecretsClient  
    UserSecretsClient().set_gcloud_credentials(project=project_id)  
else:  
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = "my_service_account_key.json"
```

```
from google.cloud import storage  
  
bucket_name = "my_bucket" ##### CHANGE THIS TO A UNIQUE BUCKET NAME #####  
location = "us-central1"  
  
storage_client = storage.Client(project=project_id)  
bucket = storage_client.create_bucket(bucket_name, location=location)  
#bucket = storage_client.bucket(bucket_name) # to reuse a bucket instead
```

```
def upload_directory(bucket, dirpath):  
    dirpath = Path(dirpath)  
    for filepath in dirpath.glob("**/*"):  
        if filepath.is_file():  
            blob = bucket.blob(filepath.relative_to(dirpath.parent).as_posix())  
            blob.upload_from_filename(filepath)  
  
upload_directory(bucket, "my_mnist_model")
```

```
# extra code - a much faster multithreaded implementation of upload_directory()
#           which also accepts a prefix for the target path, and prints stuff

from concurrent import futures

def upload_file(bucket, filepath, blob_path):
    blob = bucket.blob(blob_path)
    blob.upload_from_filename(filepath)

def upload_directory(bucket, dirpath, prefix=None, max_workers=50):
    dirpath = Path(dirpath)
    prefix = prefix or dirpath.name
    with futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
        future_to_filepath = {
            executor.submit(
                upload_file,
                bucket, filepath,
                f"{prefix}/{filepath.relative_to(dirpath).as_posix()}"
            ): filepath
            for filepath in sorted(dirpath.glob("**/*"))
            if filepath.is_file()
        }
    for future in futures.as_completed(future_to_filepath):
        filepath = future_to_filepath[future]
        try:
            result = future.result()
        except Exception as ex:
            print(f"Error uploading {filepath!s:60}: {ex}") # f!s is str(f)
        else:
            print(f"Uploaded {filepath!s:60}", end="\r")

    print(f"Uploaded {dirpath!s:60}")
```

Ngoài ra, nếu bạn đã cài đặt Google Cloud CLI (nó được cài đặt sẵn trên Colab), thì bạn có thể sử dụng lệnh `gsutil` sau:

```
!gsutil -m cp -r my_mnist_model gs://{bucket_name}/
```

```
from google.cloud import aiplatform

server_image = "gcr.io/cloud-aiplatform/prediction/tf2-gpu.2-8:latest"

aiplatform.init(project=project_id, location=location)
mnist_model = aiplatform.Model.upload(
    display_name="mnist",
    artifact_uri=f"gs://{bucket_name}/my_mnist_model/0001",
    serving_container_image_uri=server_image,
)
```

```
Creating Model
Create Model backing LRO: projects/522977795627/locations/us-central1/models/4798114811986575360/operations/53403898236370944
Model created. Resource name: projects/522977795627/locations/us-central1/models/4798114811986575360
To use this Model in another session:
model = aiplatform.Model('projects/522977795627/locations/us-central1/models/4798114811986575360')
```

Cảnh báo: ô này có thể mất vài phút để chạy, vì nó phải đợi Vertex AI cung cấp các nút tính toán:

```
endpoint = aiplatform.Endpoint.create(display_name="mnist-endpoint")

endpoint.deploy(
    mnist_model,
    min_replica_count=1,
    max_replica_count=5,
    machine_type="n1-standard-4",
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=1
)
```

```
Creating Endpoint
Create Endpoint backing LRO: projects/522977795627/locations/us-central1/endpoints/5133373499481522176/operations/41353540104943
Endpoint created. Resource name: projects/522977795627/locations/us-central1/endpoints/5133373499481522176
To use this Endpoint in another session:
endpoint = aiplatform.Endpoint('projects/522977795627/locations/us-central1/endpoints/5133373499481522176')
Deploying Model projects/522977795627/locations/us-central1/models/4798114811986575360 to Endpoint : projects/522977795627/locat
```


JobState.JOB_STATE_SUCCEEDED

BatchPredictionJob run completed. Resource name: projects/522977795627/locations/us-central1/batchPredictionJobs/434692636723799

batch_prediction_job.output_info # extra code - shows the output directory

gcs_output_directory: "gs://my_bucket/my_mnist_predictions/prediction-mnist-2022_04_12T21_30_08_071Z"

```
y_probab = []
for blob in batch_prediction_job.iter_outputs():
    print(blob.name) # extra code
    if "prediction.results" in blob.name:
        for line in blob.download_as_text().splitlines():
            y_proba = json.loads(line)["prediction"]
            y_probab.append(y_proba)
```

my_mnist_predictions/prediction-mnist-2022_04_12T21_30_08_071Z/prediction.errors_stats-00000-of-00001
my_mnist_predictions/prediction-mnist-2022_04_12T21_30_08_071Z/prediction.results-00000-of-00002
my_mnist_predictions/prediction-mnist-2022_04_12T21_30_08_071Z/prediction.results-00001-of-00002

```
y_pred = np.argmax(y_probab, axis=1)
accuracy = np.sum(y_pred == y_test[:100]) / 100
```

accuracy

0.98

mnist_model.delete()

Deleting Model : projects/522977795627/locations/us-central1/models/4798114811986575360
Delete Model backing LRO: projects/522977795627/locations/us-central1/operations/598902403101622272
Model deleted. . Resource name: projects/522977795627/locations/us-central1/models/4798114811986575360

Hãy xóa tất cả các thư mục chúng ta đã tạo trên GCS (nghĩa là tất cả các blob có các tiền tố này):

```
for prefix in ["my_mnist_model/", "my_mnist_batch/", "my_mnist_predictions/"]:
    blobs = bucket.list_blobs(prefix=prefix)
    for blob in blobs:
        blob.delete()

#bucket.delete() # uncomment and run if you want to delete the bucket itself
batch_prediction_job.delete()
```

Deleting BatchPredictionJob : projects/522977795627/locations/us-central1/batchPredictionJobs/4346926367237996544
Delete BatchPredictionJob backing LRO: projects/522977795627/locations/us-central1/operations/6699028098374959104
BatchPredictionJob deleted. . Resource name: projects/522977795627/locations/us-central1/batchPredictionJobs/4346926367237996544

✧ Triển khai Mô hình lên Thiết bị Di động hoặc Nhúng

```
converter = tf.lite.TFLiteConverter.from_saved_model(str(model_path))
tflite_model = converter.convert()
with open("my_converted_savedmodel.tflite", "wb") as f:
    f.write(tflite_model)
```

2022-04-10 09:03:52.237094: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:357] Ignored output_format.
2022-04-10 09:03:52.237108: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:360] Ignored drop_control_depend
WARNING:absl:Buffer deduplication procedure will be skipped when flatbuffer library is not properly loaded
2022-04-10 09:03:52.237830: I tensorflow/cc/saved_model/reader.cc:43] Reading SavedModel from: my_mnist_model/0001
2022-04-10 09:03:52.238869: I tensorflow/cc/saved_model/reader.cc:78] Reading meta graph with tags { serve }
2022-04-10 09:03:52.238881: I tensorflow/cc/saved_model/reader.cc:119] Reading SavedModel debug info (if present) from: my_mnist
2022-04-10 09:03:52.242108: I tensorflow/cc/saved_model/loader.cc:228] Restoring SavedModel bundle.
2022-04-10 09:03:52.263868: I tensorflow/cc/saved_model/loader.cc:212] Running initialization op on SavedModel bundle at path: m
2022-04-10 09:03:52.271298: I tensorflow/cc/saved_model/loader.cc:301] SavedModel load for tags { serve }; Status: success: OK.
2022-04-10 09:03:52.281694: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:237] disabling MLIR crash reproducer,

```
# extra code - shows how to convert a Keras model
converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

```
tflite_model = converter.convert()
with open("my_converted_keras_model.tflite", "wb") as f:
    f.write(tflite_model)
```

```
INFO:tensorflow:Assets written to: /var/folders/wy/h39t6kb11pnbb0pzhksd_fqh0000gq/T/tmp6ffbc1qs/assets
INFO:tensorflow:Assets written to: /var/folders/wy/h39t6kb11pnbb0pzhksd_fqh0000gq/T/tmp6ffbc1qs/assets
WARNING:absl:Buffer deduplication procedure will be skipped when flatbuffer library is not properly loaded
2022-04-10 09:26:30.319286: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:357] Ignored output_format.
2022-04-10 09:26:30.319301: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:360] Ignored drop_control_depend
2022-04-10 09:26:30.319417: I tensorflow/cc/saved_model/reader.cc:43] Reading SavedModel from: /var/folders/wy/h39t6kb11pnbb0pzh
2022-04-10 09:26:30.320420: I tensorflow/cc/saved_model/reader.cc:78] Reading meta graph with tags { serve }
2022-04-10 09:26:30.320431: I tensorflow/cc/saved_model/reader.cc:119] Reading SavedModel debug info (if present) from: /var/fo
2022-04-10 09:26:30.323773: I tensorflow/cc/saved_model/loader.cc:228] Restoring SavedModel bundle.
2022-04-10 09:26:30.345416: I tensorflow/cc/saved_model/loader.cc:212] Running initialization op on SavedModel bundle at path: /
2022-04-10 09:26:30.354270: I tensorflow/cc/saved_model/loader.cc:301] SavedModel load for tags { serve }; Status: success: OK.
2022-04-10 09:26:30.392352: I tensorflow/lite/tools/optimize/quantize_weights.cc:225] Skipping quantization of tensor sequential
```

✓ Chạy Mô hình trên Trang Web

Các ví dụ mã nguồn cho phần này được lưu trữ trên glitch.com, một trang web cho phép bạn tạo các ứng dụng Web miễn phí.

- <https://homl.info/tfjscode>: một ứng dụng Web TFJS đơn giản tải mô hình đã huấn luyện trước và phân loại một hình ảnh.
- <https://homl.info/tfjswpa>: cùng một thiết lập ứng dụng Web dưới dạng WPA. Hãy thử mở liên kết này trên các nền tảng khác nhau, bao gồm cả thiết bị di động. ** <https://homl.info/wpacode>: mã nguồn của WPA này.
- <https://tensorflow.org/js>: Thư viện TFJS. ** <https://www.tensorflow.org/js/demos>: một số bản demo thú vị.

✓ Sử dụng GPU để Tăng tốc Tính toán

Hãy kiểm tra xem TensorFlow có thể nhìn thấy GPU không:

```
physical_gpus = tf.config.list_physical_devices("GPU")
physical_gpus

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Nếu bạn muốn script TensorFlow của mình chỉ sử dụng GPU #0 và #1 (dựa trên thứ tự PCI), thì bạn có thể đặt các biến môi trường `CUDA_DEVICE_ORDER=PCI_BUS_ID` và `CUDA_VISIBLE_DEVICES=0,1` trước khi bắt đầu script hoặc ngay trong script trước khi sử dụng TensorFlow.

✓ Quản lý RAM của GPU

Để giới hạn lượng RAM ở mức 2GB cho mỗi GPU:

```
#for gpu in physical_gpus:
#    tf.config.set_logical_device_configuration(
#        gpu,
#        [tf.config.LogicalDeviceConfiguration(memory_limit=2048)]
#    )
```

Để TensorFlow lấy bộ nhớ khi cần (chỉ giải phóng khi tiến trình dừng):

```
#for gpu in physical_gpus:
#    tf.config.experimental.set_memory_growth(gpu, True)
```

Tương đương, bạn có thể đặt biến môi trường `TF_FORCE_GPU_ALLOW_GROWTH` thành `true` trước khi sử dụng TensorFlow.

Để chia một GPU vật lý thành hai GPU logic:

```
#tf.config.set_logical_device_configuration(
#    physical_gpus[0],
#    [tf.config.LogicalDeviceConfiguration(memory_limit=2048),
```

```
# tf.config.LogicalDeviceConfiguration(memory_limit=2048)]
#)
```

```
logical_gpus = tf.config.list_logical_devices("GPU")
logical_gpus
```

```
[LogicalDevice(name='/device:GPU:0', device_type='GPU')]
```

▼ Đặt các Phép toán và Biến lên Thiết bị

Để ghi nhật ký (log) vị trí đặt mọi biến và phép toán (điều này phải được chạy ngay sau khi nhập TensorFlow):

```
#tf.get_logger().setLevel("DEBUG") # log level is INFO by default
#tf.debugging.set_log_device_placement(True)
```

```
a = tf.Variable([1., 2., 3.]) # float32 variable goes to the GPU
a.device
```

```
'/job:localhost/replica:0/task:0/device:GPU:0'
```

```
b = tf.Variable([1, 2, 3]) # int32 variable goes to the CPU
b.device
```

```
'/job:localhost/replica:0/task:0/device:CPU:0'
```

Bạn có thể đặt các biến và phép toán theo cách thủ công trên thiết bị mong muốn bằng cách sử dụng ngữ cảnh `tf.device()`:

```
with tf.device("/cpu:0"):
    c = tf.Variable([1., 2., 3.])
```

```
c.device
```

```
'/job:localhost/replica:0/task:0/device:CPU:0'
```

Nếu bạn chỉ định một thiết bị không tồn tại, hoặc thiết bị không có kernel, TensorFlow sẽ âm thầm quay lại vị trí mặc định:

```
# extra code
```

```
with tf.device("/gpu:1234"):
    d = tf.Variable([1., 2., 3.])
```

```
d.device
```

```
''/job:localhost/replica:0/task:0/device:GPU:0''
```

Nếu bạn muốn TensorFlow đưa ra ngoại lệ khi bạn cố gắng sử dụng một thiết bị không tồn tại, thay vì quay lại thiết bị mặc định:

```
tf.config.set_soft_device_placement(False)
```

```
# extra code
```

```
try:
```

```
    with tf.device("/gpu:1000"):
        d = tf.Variable([1., 2., 3.])
except tf.errors.InvalidArgumentError as ex:
    print(ex)
```

```
tf.config.set_soft_device_placement(True) # extra code - back to soft placement
```

```
Could not satisfy device specification '/job:localhost/replica:0/task:0/device:GPU:1000'. enable_soft_placement=0. Supported dev
```

▼ Thực thi Song song trên Nhiều Thiết bị

Nếu bạn muốn đặt số lượng luồng inter-op hoặc intra-op (điều này có thể hữu ích nếu bạn muốn tránh làm bão hòa CPU, hoặc nếu bạn muốn làm cho TensorFlow chạy đơn luồng để có trường hợp kiểm thử có thể tái lập hoàn hảo):

```
#tf.config.threading.set_inter_op_parallelism_threads(10)
#tf.config.threading.set_intra_op_parallelism_threads(10)
```

- ✧ Huấn luyện Mô hình trên Nhiều Thiết bị
- ✧ Huấn luyện Quy mô lớn bằng Distribution Strategies API

```
# extra code - creates a CNN model for MNIST using Keras
def create_model():
    return tf.keras.Sequential([
        tf.keras.layers.Reshape([28, 28, 1], input_shape=[28, 28],
                                dtype=tf.uint8),
        tf.keras.layers.Rescaling(scale=1 / 255),
        tf.keras.layers.Conv2D(filters=64, kernel_size=7, activation="relu",
                                padding="same"),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation="relu",
                                padding="same"),
        tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation="relu",
                                padding="same"),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(units=64, activation="relu"),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(units=10, activation="softmax"),
    ])
```

```
tf.random.set_seed(42)

strategy = tf.distribute.MirroredStrategy()

with strategy.scope():
    model = create_model() # create a Keras model normally
    model.compile(loss="sparse_categorical_crossentropy",
                  optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
                  metrics=["accuracy"]) # compile the model normally

batch_size = 100 # preferably divisible by the number of replicas
model.fit(X_train, y_train, epochs=10,
          validation_data=(X_valid, y_valid), batch_size=batch_size)
```

```
type(model.weights[0])
```

```
tensorflow.python.distribute.values.MirroredVariable
```

```
model.predict(X_new).round(2) # extra code - the batch is split across all replicas
```

```
array([[0., 0., 0., 0., 0., 0., 0., 1., 0., 0.],
       [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.]], dtype=float32)
```

```
# extra code - shows that saving a model does not preserve its distribution
#
strategy
model.save("my_mirrored_model", save_format="tf")
model = tf.keras.models.load_model("my_mirrored_model")
type(model.weights[0])
```

```
INFO:tensorflow:Assets written to: my_mirrored_model/assets
tensorflow.python.ops.resource_variable_ops.ResourceVariable
```

```
with strategy.scope():
    model = tf.keras.models.load_model("my_mirrored_model")
```

```
type(model.weights[0])
```

```
tensorflow.python.distribute.values.MirroredVariable
```

Nếu bạn muốn chỉ định danh sách các GPU cần dùng:

```
strategy = tf.distribute.MirroredStrategy(devices=["/gpu:0", "/gpu:1"])
```

```
WARNING:tensorflow:Some requested devices in `tf.distribute.Strategy` are not visible to TensorFlow: /job:localhost/replica:0/task:0/device:GPU:0  
INFO:tensorflow:Using MirroredStrategy with devices ('/job:localhost/replica:0/task:0/device:GPU:0', '/job:localhost/replica:0/task:0/device:GPU:1')
```

Nếu bạn muốn thay đổi thuật toán all-reduce mặc định:

```
strategy = tf.distribute.MirroredStrategy(  
    cross_device_ops=tf.distribute.HierarchicalCopyAllReduce())
```

```
INFO:tensorflow:Using MirroredStrategy with devices ('/job:localhost/replica:0/task:0/device:CPU:0',)
```

Nếu bạn muốn sử dụng `CentralStorageStrategy`:

```
strategy = tf.distribute.experimental.CentralStorageStrategy()
```

```
INFO:tensorflow:ParameterServerStrategy (CentralStorageStrategy if you are using a single machine) with compute_devices = ['/job:localhost/replica:0/task:0/device:CPU:0']
```

```
# To train on a TPU in Google Colab:  
#if "google.colab" in sys.modules and "COLAB_TPU_ADDR" in os.environ:  
#    tpu_address = "grpc://" + os.environ["COLAB_TPU_ADDR"]  
#else:  
#    tpu_address = ""  
#resolver = tf.distribute.cluster_resolver.TPUClusterResolver(tpu_address)  
#tf.config.experimental_connect_to_cluster(resolver)  
#tf.tpu.experimental.initialize_tpu_system(resolver)  
#strategy = tf.distribute.experimental.TPUStrategy(resolver)
```

▼ Huấn luyện Mô hình trên một Cụm TensorFlow (Cluster)

Một cụm TensorFlow là một nhóm các tiến trình TensorFlow chạy song song, thường trên các máy khác nhau và giao tiếp với nhau để hoàn thành một công việc, ví dụ như huấn luyện hoặc thực thi một mạng thần kinh. Mỗi tiến trình TF trong cụm được gọi là một "nhiệm vụ" (task - hoặc một "TF server"). Nó có địa chỉ IP, cổng và loại (còn gọi là vai trò hoặc công việc của nó). Loại có thể là `"worker"`, `"chief"`, `"ps"` (parameter server - máy chủ tham số) hoặc `"evaluator"` (máy đánh giá):

- Mỗi **worker** thực hiện các tính toán, thường trên máy có một hoặc nhiều GPU.
- **chief** cũng thực hiện các tính toán, nhưng nó cũng xử lý các công việc bổ sung như ghi nhật ký TensorBoard hoặc lưu các điểm kiểm tra (checkpoints). Có duy nhất một chief trong một cụm. Nếu nó không được định nghĩa rõ ràng, thì theo mặc định worker #0 sẽ là chief.
- Một **parameter server** (ps) chỉ theo dõi giá trị của các biến, nó thường nằm trên máy chỉ có CPU.
- **evaluator** đảm nhận việc đánh giá. Thường có một máy đánh giá duy nhất trong một cụm.

Tập hợp các nhiệm vụ có cùng loại thường được gọi là một "job". Ví dụ, job "worker" là tập hợp của tất cả các worker.

Để bắt đầu một cụm TensorFlow, trước tiên bạn phải định nghĩa nó. Điều này có nghĩa là chỉ định tất cả các nhiệm vụ (địa chỉ IP, cổng TCP và loại). Ví dụ, cấu hình cụm sau đây định nghĩa một cụm có 3 nhiệm vụ (2 worker và 1 parameter server). Đó là một từ điển với mỗi khóa cho một job và giá trị là danh sách các địa chỉ nhiệm vụ:

```
cluster_spec = {  
    "worker": [  
        "machine-a.example.com:2222",    # /job:worker/task:0  
        "machine-b.example.com:2222"    # /job:worker/task:1  
    ],  
    "ps": ["machine-a.example.com:2221"] # /job:ps/task:0  
}
```

Mọi nhiệm vụ trong cụm có thể giao tiếp với mọi nhiệm vụ khác trong server, vì vậy hãy đảm bảo cấu hình tường lửa của bạn để cho phép tất cả các liên lạc giữa các máy này trên các cổng này (thường đơn giản hơn nếu bạn sử dụng cùng một cổng trên mỗi máy).

Khi một nhiệm vụ được khởi động, nó cần được biết nó là nhiệm vụ nào: loại và chỉ số của nó (chỉ số nhiệm vụ còn được gọi là ID nhiệm vụ). Một cách phổ biến để chỉ định mọi thứ cùng một lúc (cả cấu hình cụm và loại cũng như ID của nhiệm vụ hiện tại) là đặt biến môi trường `TF_CONFIG` trước khi bắt đầu chương trình. Nó phải là một từ điển được mã hóa JSON chứa cấu hình cụm (dưới khóa `"cluster"`), loại và chỉ số của nhiệm vụ cần bắt đầu (dưới khóa `"task"`). Ví dụ, biến môi trường `TF_CONFIG` sau đây định nghĩa cùng một cụm như trên, với 2 worker và 1 parameter server, và chỉ định rằng nhiệm vụ cần bắt đầu là worker #0:

```
os.environ["TF_CONFIG"] = json.dumps({
    "cluster": cluster_spec,
    "task": {"type": "worker", "index": 0}
})
```

Một số nền tảng (ví dụ: Google Vertex AI) tự động đặt biến môi trường này cho bạn.

Lớp `TFConfigClusterResolver` của TensorFlow đọc cấu hình cụm từ biến môi trường này:

```
resolver = tf.distribute.cluster_resolver.TFConfigClusterResolver()
resolver.cluster_spec()
```

```
ClusterSpec({'ps': ['machine-a.example.com:2221'], 'worker': ['machine-a.example.com:2222', 'machine-b.example.com:2222']})
```

```
resolver.task_type
```

```
'worker'
```

```
resolver.task_id
```

```
0
```

Bây giờ hãy chạy một cụm đơn giản hơn chỉ với hai nhiệm vụ worker, cả hai đều chạy trên máy cục bộ. Chúng ta sẽ sử dụng `MultiWorkerMirroredStrategy` để huấn luyện một mô hình trên hai nhiệm vụ này.

Bước đầu tiên là viết mã huấn luyện. Vì mã này sẽ được sử dụng để chạy cả hai worker, mỗi worker trong tiến trình riêng của nó, chúng ta viết mã này vào một tệp Python riêng biệt, `my_mnist_multiworker_task.py`. Mã này tương đối đơn giản, nhưng có một vài điều quan trọng cần lưu ý:

- Chúng ta tạo `MultiWorkerMirroredStrategy` trước khi làm bất kỳ điều gì khác với TensorFlow.
- Chỉ một trong các worker sẽ đảm nhận việc ghi nhật ký vào TensorBoard. Như đã đề cập trước đó, worker này được gọi là *chief*. Khi nó không được định nghĩa rõ ràng, theo quy ước nó là worker #0.

```
%%writefile my_mnist_multiworker_task.py
```

```
import tempfile
import tensorflow as tf
```

```
strategy = tf.distribute.MultiWorkerMirroredStrategy() # at the start!
resolver = tf.distribute.cluster_resolver.TFConfigClusterResolver()
print(f"Starting task {resolver.task_type} #{resolver.task_id}")
```

```
# extra code - Load and split the MNIST dataset
mnist = tf.keras.datasets.mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = mnist
X_valid, X_train = X_train_full[:5000], X_train_full[5000:]
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]
```

```
with strategy.scope():
    model = tf.keras.Sequential([
        tf.keras.layers.Reshape([28, 28, 1], input_shape=[28, 28],
                                   dtype=tf.uint8),
        tf.keras.layers.Rescaling(scale=1 / 255),
        tf.keras.layers.Conv2D(filters=64, kernel_size=7, activation="relu",
                                padding="same", input_shape=[28, 28, 1]),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation="relu",
                                padding="same"),
        tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation="relu",
```

```

        padding="same"),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(units=64, activation="relu"),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(units=10, activation="softmax"),
    ])
    model.compile(loss="sparse_categorical_crossentropy",
                  optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
                  metrics=["accuracy"])

    model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=10)

if resolver.task_id == 0: # the chief saves the model to the right location
    model.save("my_mnist_multiworker_model", save_format="tf")
else:
    tmpdir = tempfile.mkdtemp() # other workers save to a temporary directory
    model.save(tmpdir, save_format="tf")
    tf.io.gfile.rmtree(tmpdir) # and we can delete this directory at the end!

Writing my_mnist_multiworker_task.py

```

Trong một ứng dụng thực tế, thường sẽ có một worker duy nhất trên mỗi máy, nhưng trong ví dụ này, chúng ta đang chạy cả hai worker trên cùng một máy, vì vậy cả hai sẽ cố gắng sử dụng tất cả RAM GPU hiện có (nếu máy này có GPU) và điều này có thể dẫn đến lỗi Out-Of-Memory (OOM). Để tránh điều này, chúng ta có thể sử dụng biến môi trường `CUDA_VISIBLE_DEVICES` để gán một GPU khác nhau cho mỗi worker. Ngoài ra, chúng ta có thể đơn giản là tắt hỗ trợ GPU bằng cách đặt `CUDA_VISIBLE_DEVICES` thành một chuỗi trống.

Bây giờ chúng ta đã sẵn sàng để khởi động cả hai worker, mỗi worker trong tiến trình riêng của nó. Lưu ý rằng chúng ta thay đổi chỉ số nhiệm vụ:

```

%%bash --bg

export CUDA_VISIBLE_DEVICES=''
export TF_CONFIG='{"cluster": {"worker": ["127.0.0.1:9901", "127.0.0.1:9902"]},
                  "task": {"type": "worker", "index": 0}}'
python my_mnist_multiworker_task.py > my_worker_0.log 2>&1

```

```

%%bash --bg

export CUDA_VISIBLE_DEVICES=''
export TF_CONFIG='{"cluster": {"worker": ["127.0.0.1:9901", "127.0.0.1:9902"]},
                  "task": {"type": "worker", "index": 1}}'
python my_mnist_multiworker_task.py > my_worker_1.log 2>&1

```

Lưu ý: nếu bạn nhận được cảnh báo về `AutoShardPolicy`, bạn có thể yên tâm bỏ qua chúng. Xem [TF issue #42146](#) để biết thêm chi tiết.

Vậy là xong! Cụm TensorFlow của chúng ta hiện đang chạy, nhưng chúng ta không thể thấy nó trong notebook này vì nó đang chạy trong các tiến trình riêng biệt (nhưng bạn có thể thấy tiến trình trong `my_worker_*.log`).

Vì chief (worker #0) đang ghi vào TensorBoard, chúng ta sử dụng TensorBoard để xem tiến trình huấn luyện. Chạy ô sau, sau đó nhấp vào nút cài đặt (biểu tượng bánh răng) trong giao diện TensorBoard và tích vào ô "Reload data" để TensorBoard tự động làm mới sau mỗi 30 giây. Sau khi epoch huấn luyện đầu tiên kết thúc (có thể mất vài phút) và sau khi TensorBoard làm mới, tab SCALARS sẽ xuất hiện. Nhấp vào tab này để xem tiến trình huấn luyện mô hình và độ chính xác trên tập validation.

```

%load_ext tensorboard
%tensorboard --logdir=./my_mnist_multiworker_logs --port=6006

# strategy = tf.distribute.MultiWorkerMirroredStrategy(
#     communication_options=tf.distribute.experimental.CommunicationOptions(
#         implementation=tf.distribute.experimental.CollectiveCommunication.NCCL))

```

✓ Chạy các Công việc Huấn luyện Lớn trên Vertex AI

Hãy sao chép script huấn luyện, nhưng thêm `import os` và thay đổi đường dẫn lưu thành đường dẫn GCS mà biến môi trường `AIP_MODEL_DIR` sẽ trỏ tới:

```

%%writefile my_vertex_ai_training_task.py

import os
from pathlib import Path
import tempfile
import tensorflow as tf

strategy = tf.distribute.MultiWorkerMirroredStrategy() # at the start!
resolver = tf.distribute.cluster_resolver.TFConfigClusterResolver()

if resolver.task_type == "chief":
    model_dir = os.getenv("AIP_MODEL_DIR") # paths provided by Vertex AI
    tensorboard_log_dir = os.getenv("AIP_TENSORBOARD_LOG_DIR")
    checkpoint_dir = os.getenv("AIP_CHECKPOINT_DIR")
else:
    tmp_dir = Path(tempfile.mkdtemp()) # other workers use a temporary dirs
    model_dir = tmp_dir / "model"
    tensorboard_log_dir = tmp_dir / "logs"
    checkpoint_dir = tmp_dir / "ckpt"

callbacks = [tf.keras.callbacks.TensorBoard(tensorboard_log_dir),
             tf.keras.callbacks.ModelCheckpoint(checkpoint_dir)]

# extra code - Load and prepare the MNIST dataset
mnist = tf.keras.datasets.mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = mnist
X_valid, X_train = X_train_full[:5000], X_train_full[5000:]
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]

# extra code - build and compile the Keras model using the distribution strategy
with strategy.scope():
    model = tf.keras.Sequential([
        tf.keras.layers.Reshape([28, 28, 1], input_shape=[28, 28],
                                dtype=tf.uint8),
        tf.keras.layers.Lambda(lambda X: X / 255),
        tf.keras.layers.Conv2D(filters=64, kernel_size=7, activation="relu",
                                padding="same", input_shape=[28, 28, 1]),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation="relu",
                                padding="same"),
        tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation="relu",
                                padding="same"),
        tf.keras.layers.MaxPooling2D(pool_size=2),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(units=64, activation="relu"),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(units=10, activation="softmax"),
    ])
    model.compile(loss="sparse_categorical_crossentropy",
                  optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
                  metrics=["accuracy"])

model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=10,
          callbacks=callbacks)
model.save(model_dir, save_format="tf")

```

Writing my_vertex_ai_training_task.py

```

custom_training_job = aiplatform.CustomTrainingJob(
    display_name="my_custom_training_job",
    script_path="my_vertex_ai_training_task.py",
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-4:latest",
    model_serving_container_image_uri=server_image,
    requirements=["gcsfs==2022.3.0"], # not needed, this is just an example
    staging_bucket=f"gs://{bucket_name}/staging"
)

```

```

mnist_model2 = custom_training_job.run(
    machine_type="n1-standard-4",
    replica_count=2,
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2,
)

```



```

Training script copied to:
gs://my_bucket/aiplatform-2022-04-14-10:08:24.124-aiplatform_custom_trainer_script-0.1.tar.gz.
Training Output directory:
gs://my_bucket/aiplatform-custom-training-2022-04-14-10:08:25.226
View Training:
https://console.cloud.google.com/ai/platform/locations/us-central1/training/5407999068506947584?project=522977795627
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/5407999068506947584 current state:
PipelineState.PIPELINE_STATE_PENDING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/5407999068506947584 current state:
PipelineState.PIPELINE_STATE_RUNNING
View backing custom job:
https://console.cloud.google.com/ai/platform/locations/us-central1/training/6685701948726837248?project=522977795627
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/5407999068506947584 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/5407999068506947584 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/5407999068506947584 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/5407999068506947584 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/5407999068506947584 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/5407999068506947584 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob run completed. Resource name: projects/522977795627/locations/us-central1/trainingPipelines/540799906850694758
Model available at projects/522977795627/locations/us-central1/models/9094548856498028544

```

Hãy dọn dẹp:

```

mnist_model2.delete()
custom_training_job.delete()
blobs = bucket.list_blobs(prefix=f"gs://{bucket_name}/staging/")
for blob in blobs:
    blob.delete()

```

✧ Tinh chỉnh Siêu tham số (Hyperparameter Tuning) trên Vertex AI

```

%%writefile my_vertex_ai_trial.py

import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--n_hidden", type=int, default=2)
parser.add_argument("--n_neurons", type=int, default=256)
parser.add_argument("--learning_rate", type=float, default=1e-2)
parser.add_argument("--optimizer", default="adam")
args = parser.parse_args()

import tensorflow as tf

def build_model(args):
    with tf.distribute.MirroredStrategy().scope():
        model = tf.keras.Sequential()
        model.add(tf.keras.layers.Flatten(input_shape=[28, 28], dtype=tf.uint8))
        for _ in range(args.n_hidden):
            model.add(tf.keras.layers.Dense(args.n_neurons, activation="relu"))
        model.add(tf.keras.layers.Dense(10, activation="softmax"))
        opt = tf.keras.optimizers.get(args.optimizer)
        opt.learning_rate = args.learning_rate
        model.compile(loss="sparse_categorical_crossentropy", optimizer=opt,
                      metrics=["accuracy"])
        return model

# extra code - loads and splits the dataset
mnist = tf.keras.datasets.mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = mnist
X_valid, X_train = X_train_full[:5000], X_train_full[5000:]
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]

# extra code - use the AIP_* environment variable and create the callbacks
import os
model_dir = os.getenv("AIP_MODEL_DIR")
tensorboard_log_dir = os.getenv("AIP_TENSORBOARD_LOG_DIR")
checkpoint_dir = os.getenv("AIP_CHECKPOINT_DIR")

```

```

trial_id = os.getenv("CLOUD_ML_TRIAL_ID")
tensorboard_cb = tf.keras.callbacks.TensorBoard(tensorboard_log_dir)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=5)
callbacks = [tensorboard_cb, early_stopping_cb]

model = build_model(args)
history = model.fit(X_train, y_train, validation_data=(X_valid, y_valid),
                    epochs=10, callbacks=callbacks)
model.save(model_dir, save_format="tf") # extra code

import hypertune

hypertune = hypertune.HyperTune()
hypertune.report_hyperparameter_tuning_metric(
    hyperparameter_metric_tag="accuracy", # name of the reported metric
    metric_value=max(history.history["val_accuracy"]), # max accuracy value
    global_step=model.optimizer.iterations.numpy(),
)

```

Writing my_vertex_ai_trial.py

```

trial_job = aiplatform.CustomJob.from_local_script(
    display_name="my_search_trial_job",
    script_path="my_vertex_ai_trial.py", # path to your training script
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-4:latest",
    staging_bucket=f"gs://{bucket_name}/staging",
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2, # in this example, each trial will have 2 GPUs
)

```

Training script copied to:
gs://hom13-mybucket5/staging/aiplatform-2022-04-18-18:14:02.860-aiplatform_custom_trainer_script-0.1.tar.gz.

```

from google.cloud.aiplatform import hyperparameter_tuning as hpt

hp_job = aiplatform.HyperparameterTuningJob(
    display_name="my_hp_search_job",
    custom_job=trial_job,
    metric_spec={"accuracy": "maximize"},
    parameter_spec={
        "learning_rate": hpt.DoubleParameterSpec(min=1e-3, max=10, scale="log"),
        "n_neurons": hpt.IntegerParameterSpec(min=1, max=300, scale="linear"),
        "n_hidden": hpt.IntegerParameterSpec(min=1, max=10, scale="linear"),
        "optimizer": hpt.CategoricalParameterSpec(["sgd", "adam"]),
    },
    max_trial_count=100,
    parallel_trial_count=20,
)
hp_job.run()

```

Creating HyperparameterTuningJob
HyperparameterTuningJob created. Resource name: projects/522977795627/locations/us-central1/hyperparameterTuningJobs/58251361878
To use this HyperparameterTuningJob in another session:
hpt_job = aiplatform.HyperparameterTuningJob.get('projects/522977795627/locations/us-central1/hyperparameterTuningJobs/58251361878')
View HyperparameterTuningJob:
<https://console.cloud.google.com/ai/platform/locations/us-central1/training/5825136187899117568?project=522977795627>
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_RUNNING
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_RUNNING
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_RUNNING
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_RUNNING
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_RUNNING
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_RUNNING
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_RUNNING
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_RUNNING
HyperparameterTuningJob projects/522977795627/locations/us-central1/hyperparameterTuningJobs/5825136187899117568 current state: JobState.JOB_STATE_SUCCEEDED
HyperparameterTuningJob run completed. Resource name: projects/522977795627/locations/us-central1/hyperparameterTuningJobs/58251

```
def get_final_metric(trial, metric_id):
    for metric in trial.final_measurement.metrics:
        if metric.metric_id == metric_id:
            return metric.value

trials = hp_job.trials
trial accuracies = [get_final_metric(trial, "accuracy") for trial in trials]
best_trial = trials[np.argmax(trial accuracies)]
```

```
max(trial accuracies)
```

```
0.977400004863739
```

```
best_trial.id
```

```
'98'
```

```
best_trial.parameters
```

```
[parameter_id: "learning_rate"
value {
  number_value: 0.001
}
, parameter_id: "n_hidden"
value {
  number_value: 8.0
}
, parameter_id: "n_neurons"
value {
  number_value: 216.0
}
, parameter_id: "optimizer"
value {
  string_value: "adam"
}
]
```

▼ Tài liệu Bổ sung – Keras Tuner Phân tán trên Vertex AI

Thay vì sử dụng dịch vụ tính toán siêu tham số của Vertex AI, bạn có thể sử dụng [Keras Tuner](#) (đã giới thiệu trong Chương 10) và chạy nó trên các máy ảo Vertex AI. Keras Tuner cung cấp một cách đơn giản để mở rộng quy mô tìm kiếm siêu tham số bằng cách phân phối nó trên nhiều máy: nó chỉ yêu cầu thiết lập ba biến môi trường trên mỗi máy, sau đó chạy mã Keras Tuner thông thường của bạn trên mỗi máy. Bạn có thể sử dụng cùng một script trên tất cả các máy. Một trong các máy đóng vai trò là chief, và các máy khác đóng vai trò là worker. Mỗi worker hỏi chief xem nên thử các giá trị siêu tham số nào—nó đóng vai trò như một oracle (nhà tiên tri)—sau đó worker huấn luyện mô hình bằng các giá trị siêu tham số này, và cuối cùng nó báo cáo hiệu suất của mô hình cho chief, từ đó chief có thể quyết định các giá trị siêu tham số mà worker nên thử tiếp theo.

Ba biến môi trường bạn cần thiết lập trên mỗi máy là:

- `KERASTUNER_TUNER_ID`: bằng `"chief"` trên máy chief, hoặc một định danh duy nhất trên mỗi máy worker, chẳng hạn như `"worker0"`, `"worker1"`, v.v.
- `KERASTUNER_ORACLE_IP`: địa chỉ IP hoặc hostname của máy chief. Bản thân chief thường nên sử dụng `"0.0.0.0"` để lắng nghe trên mọi địa chỉ IP của máy.
- `KERASTUNER_ORACLE_PORT`: cổng TCP mà chief sẽ lắng nghe.

Bạn có thể sử dụng Keras Tuner phân tán trên bất kỳ tập hợp máy nào. Nếu bạn muốn chạy nó trên các máy Vertex AI, thì bạn có thể tạo một công việc huấn luyện thông thường và chỉ cần sửa đổi script huấn luyện để thiết lập ba biến môi trường đúng cách trước khi sử dụng Keras Tuner.

Ví dụ, script bên dưới bắt đầu bằng cách phân tích biến môi trường `TF_CONFIG`, biến này sẽ được Vertex AI tự động thiết lập, giống như trước đây. Nó tìm địa chỉ của nhiệm vụ loại `"chief"`, trích xuất địa chỉ IP hoặc hostname và cổng TCP. Sau đó, nó định nghĩa ID của tuner là loại nhiệm vụ theo sau là chỉ số nhiệm vụ, ví dụ `"worker0"`. Nếu ID tuner là `"chief0"`, nó sẽ đổi thành `"chief"` và đặt IP thành `"0.0.0.0"`: điều này sẽ làm nó lắng nghe trên tất cả các địa chỉ IPv4 trên máy của nó. Tiếp theo, nó định nghĩa các biến môi trường cho Keras Tuner. Sau đó, script tạo một tuner, giống như trong Chương 10, chạy tìm kiếm, và cuối cùng lưu mô hình tốt nhất vào vị trí do Vertex AI cung cấp:

```
%%writefile my_keras_tuner_search.py
```

```

import json
import os

tf_config = json.loads(os.environ["TF_CONFIG"])

chief_ip, chief_port = tf_config["cluster"]["chief"][0].rsplit(":", 1)
tuner_id = f'{tf_config["task"]["type"]}{tf_config["task"]["index"]}'
if tuner_id == "chief0":
    tuner_id = "chief"
    chief_ip = "0.0.0.0"
    # extra code - since the chief doesn't work much, you can optimize compute
    # resources by running a worker on the same machine. To do this, you can
    # just make the chief start another process, after tweaking the TF_CONFIG
    # environment variable to set the task type to "worker" and the task index
    # to a unique value. Uncomment the next few lines to give this a try:
    # import subprocess
    # import sys
    # tf_config["task"]["type"] = "workerX" # the worker on the chief's machine
    # os.environ["TF_CONFIG"] = json.dumps(tf_config)
    # subprocess.Popen([sys.executable] + sys.argv,
    #                   stdout=sys.stdout, stderr=sys.stderr)

os.environ["KERASTUNER_TUNER_ID"] = tuner_id
os.environ["KERASTUNER_ORACLE_IP"] = chief_ip
os.environ["KERASTUNER_ORACLE_PORT"] = chief_port

from pathlib import Path
import keras_tuner as kt
import tensorflow as tf

gcs_path = "/gcs/my_bucket/my_hp_search" # replace with your bucket's name

def build_model(hp):
    n_hidden = hp.Int("n_hidden", min_value=0, max_value=8, default=2)
    n_neurons = hp.Int("n_neurons", min_value=16, max_value=256)
    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2,
                             sampling="log")
    optimizer = hp.Choice("optimizer", values=["sgd", "adam"])
    if optimizer == "sgd":
        optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)
    else:
        optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)

    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Flatten(input_shape=[28, 28], dtype=tf.uint8))
    for _ in range(n_hidden):
        model.add(tf.keras.layers.Dense(n_neurons, activation="relu"))
    model.add(tf.keras.layers.Dense(10, activation="softmax"))
    model.compile(loss="sparse_categorical_crossentropy",
                  optimizer=optimizer,
                  metrics=["accuracy"])

    return model

hyperband_tuner = kt.Hyperband(
    build_model, objective="val_accuracy", seed=42,
    max_epochs=10, factor=3, hyperband_iterations=2,
    distribution_strategy=tf.distribute.MirroredStrategy(),
    directory=gcs_path, project_name="mnist")

# extra code - Load and split the MNIST dataset
mnist = tf.keras.datasets.mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = mnist
X_valid, X_train = X_train_full[:5000], X_train_full[5000:]
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]

tensorboard_log_dir = os.environ["AIP_TENSORBOARD_LOG_DIR"] + "/" + tuner_id
tensorboard_cb = tf.keras.callbacks.TensorBoard(tensorboard_log_dir)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=5)
hyperband_tuner.search(X_train, y_train, epochs=10,
                      validation_data=(X_valid, y_valid),
                      callbacks=[tensorboard_cb, early_stopping_cb])

if tuner_id == "chief":
    best_hp = hyperband_tuner.get_best_hyperparameters()[0]
    best_model = hyperband_tuner.hypermodel.build(best_hp)
    best_model.save(os.getenv("AIP_MODEL_DIR"), save_format="tf")

```

Writing my_keras_tuner_search.py

Lưu ý rằng Vertex AI tự động gắn thư mục `/gcs` vào GCS, sử dụng mã nguồn mở [GCS Fuse adapter](#). Điều này mang lại cho chúng ta một thư mục dùng chung giữa các worker và chief, điều bắt buộc đối với Keras Tuner. Cũng lưu ý rằng chúng ta đặt chiến lược phân phối là `MirroredStrategy`. Điều này sẽ cho phép mỗi worker sử dụng tất cả các GPU trên máy của nó, nếu có nhiều hơn một GPU.

Thay thế `/gcs/my_bucket/` bằng `/gcs/{bucket_name}/`:

```
with open("my_keras_tuner_search.py") as f:
    script = f.read()

with open("my_keras_tuner_search.py", "w") as f:
    f.write(script.replace("/gcs/my_bucket/", f"/gcs/{bucket_name}/"))
```

Bây giờ tất cả những gì chúng ta cần làm là khởi động một công việc huấn luyện tùy chỉnh dựa trên script này, giống hệt như trong phần trước. Đừng quên thêm `keras-tuner` vào danh sách `requirements`:

```
hp_search_job = aiplatform.CustomTrainingJob(
    display_name="my_hp_search_job",
    script_path="my_keras_tuner_search.py",
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-4:latest",
    model_serving_container_image_uri=server_image,
    requirements=["keras-tuner~=1.1.2"],
    staging_bucket=f"gs://{bucket_name}/staging",
)
```

```
mnist_model3 = hp_search_job.run(
    machine_type="n1-standard-4",
    replica_count=3,
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2,
)
```

[illegible]

```
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/8601543785521872896 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/8601543785521872896 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/8601543785521872896 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/8601543785521872896 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob projects/522977795627/locations/us-central1/trainingPipelines/8601543785521872896 current state:
PipelineState.PIPELINE_STATE_RUNNING
CustomTrainingJob run completed. Resource name: projects/522977795627/locations/us-central1/trainingPipelines/860154378552187289
Model available at projects/522977795627/locations/us-central1/models/8176544832480168612
```

Và chúng ta đã có một mô hình!

Hãy dọn dẹp:

```
mnist_model3.delete()
hp_search_job.delete()
blobs = bucket.list_blobs(prefix=f"gs://{bucket_name}/staging/")
for blob in blobs:
    blob.delete()
```

▼ Tài liệu Bổ sung – Sử dụng AutoML để Huấn luyện một Mô hình

Hãy bắt đầu bằng cách xuất tập dữ liệu MNIST sang các ảnh PNG, và chuẩn bị một tệp `import.csv` trỏ đến từng ảnh, chỉ định phần chia (training, validation hoặc test) và nhãn:

```
import matplotlib.pyplot as plt

mnist_path = Path("datasets/mnist")
mnist_path.mkdir(parents=True, exist_ok=True)
idx = 0
with open(mnist_path / "import.csv", "w") as import_csv:
    for split, X, y in zip(("training", "validation", "test"),
                          (X_train, X_valid, X_test),
                          (y_train, y_valid, y_test)):
        for image, label in zip(X, y):
            print(f"\r{idx + 1}/70000", end="")
            filename = f"{idx:05d}.png"
            plt.imsave(mnist_path / filename, np.tile(image, 3))
            line = f"{split},gs://{bucket_name}/mnist/{filename},{label}\n"
            import_csv.write(line)
            idx += 1
```

70000/70000

Hãy tải tập dữ liệu này lên GCS:

```
upload_directory(bucket, mnist_path)
```

Uploaded datasets/mnist

Bây giờ hãy tạo một tập dữ liệu hình ảnh được quản lý trên Vertex AI:

```
from aiplatform.schema.dataset.ioformat.image import single_label_classification

mnist_dataset = aiplatform.ImageDataset.create(
    display_name="mnist-dataset",
    gcs_source=[f"gs://{bucket_name}/mnist/import.csv"],
    project=project_id,
    import_schema_uri=single_label_classification,
    sync=True,
)
```

```
Creating ImageDataset
Create ImageDataset backing LR0: projects/522977795627/locations/us-central1/datasets/7532459492777132032/operations/38122339313
ImageDataset created. Resource name: projects/522977795627/locations/us-central1/datasets/7532459492777132032
```

To use this ImageDataset in another session:

```
ds = aiplatform.ImageDataset('projects/522977795627/locations/us-central1/datasets/7532459492777132032')
```

```
Importing ImageDataset data: projects/522977795627/locations/us-central1/datasets/7532459492777132032
```

```
Import ImageDataset data backing LRO: projects/522977795627/locations/us-central1/datasets/7532459492777132032/operations/301059
```

```
ImageDataset data imported. Resource name: projects/522977795627/locations/us-central1/datasets/7532459492777132032
```

Tạo một công việc huấn luyện AutoML trên tập dữ liệu này:

TODO

↳ Lời giải Bài tập

↳ 1. to 8.

1. Một SavedModel chứa một mô hình TensorFlow, bao gồm kiến trúc của nó (một đồ thị tính toán) và trọng số của nó. Nó được lưu trữ dưới dạng một thư mục chứa tệp *saved_model.pb*, định nghĩa đồ thị tính toán (được biểu diễn dưới dạng protocol buffer được tuần tự hóa) và một thư mục con *variables* chứa các giá trị biến. Đối với các mô hình chứa số lượng lớn trọng số, các giá trị biến này có thể được chia thành nhiều tệp. SavedModel cũng bao gồm một thư mục con *assets* có thể chứa dữ liệu bổ sung, chẳng hạn như các tệp từ vựng, tên lớp hoặc một số thực thể ví dụ cho mô hình này. Để chính xác hơn, một SavedModel có thể chứa một hoặc nhiều *metagraph*. Một metagraph là một đồ thị tính toán cộng với một số định nghĩa chữ ký hàm (bao gồm tên, loại và hình dạng đầu vào và đầu ra của chúng). Mỗi metagraph được xác định bởi một tập hợp các thẻ (tags). Để kiểm tra SavedModel, bạn có thể sử dụng công cụ dòng lệnh `saved_model_cli` hoặc chỉ cần tải nó bằng `tf.saved_model.load()` và kiểm tra trong Python.
2. TF Serving cho phép bạn triển khai nhiều mô hình TensorFlow (hoặc nhiều phiên bản của cùng một mô hình) và làm cho chúng có thể truy cập được cho tất cả các ứng dụng của bạn một cách dễ dàng thông qua REST API hoặc gRPC API. Việc sử dụng trực tiếp các mô hình trong ứng dụng sẽ gây khó khăn cho việc triển khai phiên bản mới của mô hình trên tất cả các ứng dụng. Việc tự triển khai microservice để bao bọc mô hình TF sẽ tốn thêm công sức và khó có thể sánh được với các tính năng của TF Serving. TF Serving có nhiều tính năng: nó có thể giám sát một thư mục và tự động triển khai các mô hình được đặt ở đó, và bạn sẽ không phải thay đổi hoặc thậm chí khởi động lại bất kỳ ứng dụng nào của mình để hưởng lợi từ các phiên bản mô hình mới; nó nhanh, được kiểm thử kỹ lưỡng và mở rộng quy mô rất tốt; nó hỗ trợ thử nghiệm A/B các mô hình thử nghiệm và triển khai phiên bản mô hình mới chỉ cho một tập con người dùng của bạn (trong trường hợp này mô hình được gọi là *canary*). TF Serving cũng có khả năng nhóm các yêu cầu riêng lẻ thành các lô (batches) để chạy chúng cùng nhau trên GPU. Để triển khai TF Serving, bạn có thể cài đặt từ nguồn, nhưng đơn giản hơn nhiều là cài đặt nó bằng Docker image. Để triển khai một cụm các Docker image TF Serving, bạn có thể sử dụng công cụ điều phối như Kubernetes hoặc sử dụng giải pháp được lưu trữ hoàn toàn như Google Vertex AI.
3. Để triển khai một mô hình trên nhiều phiên bản TF Serving, tất cả những gì bạn cần làm là định cấu hình các phiên bản TF Serving này để giám sát cùng một thư mục *models*, sau đó xuất mô hình mới của bạn dưới dạng SavedModel vào một thư mục con.
4. gRPC API hiệu quả hơn REST API. Tuy nhiên, các thư viện client của nó không phổ biến bằng, và nếu bạn kích hoạt nén khi sử dụng REST API, bạn có thể đạt được hiệu suất gần như tương đương. Vì vậy, gRPC API hữu ích nhất khi bạn cần hiệu suất cao nhất có thể và các client không bị giới hạn ở REST API.
5. Để giảm kích thước mô hình để nó có thể chạy trên thiết bị di động hoặc nhúng, TFLite sử dụng một số kỹ thuật:
 - Nó cung cấp một bộ chuyển đổi có thể tối ưu hóa SavedModel: nó thu nhỏ mô hình và giảm độ trễ của nó. Để làm điều này, nó cắt bỏ tất cả các hoạt động không cần thiết để đưa ra dự đoán (chẳng hạn như các hoạt động huấn luyện), đồng thời tối ưu hóa và hợp nhất các hoạt động bất cứ khi nào có thể.
 - Bộ chuyển đổi cũng có thể thực hiện định lượng sau huấn luyện (post-training quantization): kỹ thuật này làm giảm đáng kể kích thước của mô hình, vì vậy nó nhanh hơn nhiều khi tải xuống và lưu trữ.
 - Nó lưu mô hình đã tối ưu hóa bằng định dạng FlatBuffer, có thể được tải trực tiếp vào RAM mà không cần phân tích cú pháp. Điều này làm giảm thời gian tải và dung lượng bộ nhớ.
6. Huấn luyện nhận biết định lượng (Quantization-aware training) bao gồm việc thêm các hoạt động định lượng giả vào mô hình trong quá trình huấn luyện. Điều này cho phép mô hình học cách bỏ qua nhiễu định lượng; trọng số cuối cùng sẽ mạnh mẽ hơn với việc định lượng.
7. Song song mô hình (Model parallelism) có nghĩa là chia mô hình của bạn thành nhiều phần và chạy chúng song song trên nhiều thiết bị, hy vọng sẽ tăng tốc mô hình trong quá trình huấn luyện hoặc suy luận. Song song dữ liệu (Data parallelism) có nghĩa là tạo ra nhiều bản sao chính xác của mô hình và triển khai chúng trên nhiều thiết bị. Tại mỗi lần lặp trong quá trình huấn luyện, mỗi bản sao được cung cấp một lô dữ liệu khác nhau và nó tính toán các gradient của hàm mất mát đối với các tham số mô hình. Trong song song dữ liệu đồng bộ, các gradient từ tất cả các bản sao sau đó được tổng hợp và trình tối ưu hóa thực hiện một bước Gradient Descent. Các tham số có thể được tập trung (ví dụ: trên các máy chủ tham số) hoặc được sao chép trên tất cả các bản sao và được

giữ đồng bộ bằng AllReduce. Trong song song dữ liệu không đồng bộ, các tham số được tập trung và các bản sao chạy độc lập với nhau, mỗi bản sao cập nhật trực tiếp các tham số trung tâm vào cuối mỗi lần lặp huấn luyện mà không cần phải đợi các bản sao khác. Để tăng tốc độ huấn luyện, song song dữ liệu tỏ ra hiệu quả hơn song song mô hình nói chung. Điều này chủ yếu là vì nó yêu cầu ít giao tiếp giữa các thiết bị hơn. Hơn nữa, nó dễ triển khai hơn nhiều và hoạt động giống nhau cho bất kỳ mô hình nào, trong khi song song mô hình yêu cầu phân tích mô hình để xác định cách tốt nhất để chia nhỏ nó. Tuy nhiên, nghiên cứu trong lĩnh vực này đang tiến triển nhanh chóng (ví dụ: PipeDream hoặc Pathways), vì vậy sự kết hợp giữa song song mô hình và song song dữ liệu có lẽ là hướng đi trong tương lai.

8. Khi huấn luyện một mô hình trên nhiều máy chủ, bạn có thể sử dụng các chiến lược phân phối sau:

- `MultiWorkerMirroredStrategy` thực hiện song song dữ liệu nhân bản. Mô hình được sao chép trên tất cả các máy chủ và thiết bị hiện có, mỗi bản sao nhận được một lô dữ liệu khác nhau tại mỗi lần lặp huấn luyện và tính toán các gradient của riêng nó. Giá trị trung bình của các gradient được tính toán và chia sẻ trên tất cả các bản sao bằng cách sử dụng triển khai AllReduce phân tán (mặc định là NCCL) và tất cả các bản sao thực hiện cùng một bước Gradient Descent. Chiến lược này là đơn giản nhất để sử dụng vì tất cả các máy chủ và thiết bị đều được đối xử theo cách hoàn toàn giống nhau và nó hoạt động khá tốt. Nhìn chung, bạn nên sử dụng chiến lược này. Hạn chế chính của nó là yêu cầu mô hình phải khớp với RAM trên mọi bản sao.
- `ParameterServerStrategy` thực hiện song song dữ liệu không đồng bộ. Mô hình được sao chép trên tất cả các thiết bị trên tất cả các worker và các tham số được chia nhỏ (sharded) trên tất cả các máy chủ tham số. Mỗi worker có vòng lặp huấn luyện riêng, chạy không đồng bộ với các worker khác; tại mỗi lần lặp huấn luyện, mỗi worker lấy lô dữ liệu của riêng mình và tìm nạp phiên bản mới nhất của các tham số mô hình từ các máy chủ tham số, sau đó nó tính toán các gradient của hàm mất mát đối với các tham số này và gửi chúng đến các máy chủ tham số. Cuối cùng, các máy chủ tham số thực hiện một bước Gradient Descent bằng cách sử dụng các gradient này. Chiến lược này thường chậm hơn chiến lược trước đó và khó triển khai hơn một chút vì nó yêu cầu quản lý các máy chủ tham số. Tuy nhiên, nó có thể hữu ích trong một số tình huống, đặc biệt là khi bạn có thể tận dụng các bản cập nhật không đồng bộ, ví dụ để giảm nghẽn cổ chai I/O. Điều này phụ thuộc vào nhiều yếu tố, bao gồm phần cứng, cấu trúc liên kết mạng, số lượng máy chủ, kích thước mô hình và hơn thế nữa.

9.

Bài tập: Huấn luyện một mô hình (bất kỳ mô hình nào bạn thích) và triển khai nó lên TF Serving hoặc Google Vertex AI. Viết mã client để truy vấn nó bằng REST API hoặc gRPC API. Cập nhật mô hình và triển khai phiên bản mới. Mã client của bạn bây giờ sẽ truy vấn phiên bản mới. Quay lại phiên bản đầu tiên.

Vui lòng làm theo các bước trong phần [Triển khai các mô hình TensorFlow lên TensorFlow Serving](#) ở trên.

10.

Bài tập: Huấn luyện bất kỳ mô hình nào trên nhiều GPU trên cùng một máy bằng `MirroredStrategy` (nếu bạn không có quyền truy cập GPU, bạn có thể sử dụng Colaboratory với GPU Runtime và tạo hai GPU ảo). Huấn luyện lại mô hình bằng `CentralStorageStrategy` và so sánh thời gian huấn luyện.

Vui lòng làm theo các bước trong phần [Huấn luyện phân tán](#) ở trên.

11.

Bài tập: Huấn luyện một mô hình nhỏ trên Google Vertex AI, sử dụng TensorFlow Cloud Tuner để tinh chỉnh siêu tham số.

Vui lòng làm theo hướng dẫn trong phần *Tinh chỉnh siêu tham số bằng TensorFlow Cloud Tuner* trong sách.

Chúc mừng!

— * * * —